SVKM'S
NMIMS
Deemed to be UNIVERSITY

Narsee Monjee Institute of Management Studies

SUBJECT: IMAGE AND VIDEO PROCESSING

Title: InSight: Intelligent Vision-Based Air-Canvas, Gesture and Image Processing System

Session: 2026-2027

Submitted By:-

Ananya Vijay

70022400132---F020

Leesa Agrawal

70022400130---F019

Srishti Sinam

70022400752---F152

B.Tech C.E.

Section - A

3nd year 5th sem

Submitted To:-

Dr. RajGaurav Mishra

Abstract

Conventional human–computer interaction relies on physical devices such as a mouse, keyboard or remote control, which is inconvenient in touch-free situations such as classrooms, presentations and public displays. This project presents **InSight**, a real-time, browser-based system that lets a user draw in the air, control media and apply image filters using only hand gestures captured by an ordinary webcam. The proposed solution processes each video frame in two parallel paths: a pixel-processing path and a hand-tracking path. Techniques used include frame extraction, grayscale conversion, thresholding, negative, sepia and contrast transforms, salt-and-pepper noise modelling, pixel-matrix inspection, hand-landmark detection with MediaPipe, exponential smoothing of the fingertip trajectory, rule-based gesture classification, and optical character recognition (Tesseract.js) of the air-written word. The recognised word is used to fetch a matching image from Wikimedia Commons. The system runs at interactive frame rates on a standard laptop without any training dataset or special hardware, and demonstrates that classical image-processing operations combined with pretrained landmark detection can deliver an intuitive, low-cost, touch-free interface.

1. Introduction

Background. Computer vision allows machines to interpret visual information from cameras. Hand gesture recognition is an active area of vision research because the hand is a natural, expressive means of communication. Air-writing—writing characters or words by moving a finger in free space—is a related problem that removes the need for a pen, touch screen or keyboard. Traditional approaches used colour-based skin segmentation, contour extraction and convexity defects, which are sensitive to lighting and background; modern approaches use learned models that locate hand keypoints directly.

Relevance. Touch-free interaction is useful in many settings: teaching with virtual whiteboards, hygienic control of public kiosks and hospital equipment, accessibility support for people who cannot use conventional input devices, and media or presentation control from a distance. Since almost every laptop and phone already contains a camera, a vision-based interface needs no extra hardware.

Role of image and video processing. A webcam supplies a video stream, which is a sequence of images. Frame extraction, colour-space conversion, thresholding and filtering prepare and analyse these frames; feature extraction (hand landmarks) converts pixels into a compact geometric description; temporal smoothing reduces jitter between frames; and the fingertip trajectory is rasterised into a binary-like image that can be processed by an OCR engine. Each stage of the Image and Video Processing syllabus therefore appears naturally in the system.

Motivation. The group chose this project because it combines many course topics in one working application, can be demonstrated live on any laptop, and has clear practical value. It also allowed the team to visualise the pixel-level meaning of image operations through the built-in pixel-matrix inspector.

2. Problem Statement and Objectives

Real-world problem. Users need a simple, low-cost, touch-free way to write or draw on a screen, control media playback and explore image-processing effects without a mouse, stylus, special glove or depth sensor. The problem is to detect the hand in a live video, understand its gesture, and map it to useful actions in real time using only a standard webcam.

Aim. To design and implement a real-time vision-based system that recognises hand gestures from webcam video and uses them to draw on a virtual canvas, recognise the written word, control a media player and switch live image filters.

Specific objectives

1. To capture live video from a webcam, extract frames and apply basic image-processing operations (grayscale, thresholding, negative, sepia, contrast, noise) in real time.
2. To detect and track the hand using 21 landmarks and to classify at least nine gestures (point, peace, pinch, fist, open palm, thumbs up/down, three and four fingers).
3. To build an **air-canvas** that converts the smoothed fingertip trajectory into strokes with colour selection, erasing and clearing.
4. To recognise the air-written word using OCR and retrieve a related image automatically.
5. To provide a pixel-matrix inspector that shows the numerical pixel values of a selected 16 x 16 region for educational visualisation.

3. Scope of the Project

What the project covers

- Real-time hand tracking for a single hand using a webcam.
- Gesture-based drawing, erasing, colour cycling and canvas clearing (Draw mode).
- Gesture-based media-player control and live filter switching (Remote mode).
- Pixel-level filters and the 16 x 16 pixel-matrix viewer with R, G, B, grayscale, binary and noise channels.
- OCR of the air-written word and automatic image lookup; saving the drawing as a PNG file.

What it does not cover

- Multi-hand or multi-user interaction (the model is configured for one hand).
- Dynamic (motion-based) gestures such as swipes or letter-shape trajectories; only static hand poses are classified.
- Training of a new deep-learning model; a pretrained hand-landmark model is used as-is.

- Native mobile app deployment, system-wide mouse control, or offline operation (CDN resources are required).

- Robust recognition of cursive handwriting or whole sentences.

Type of images / videos. Live RGB video from a front-facing webcam, processed frame by frame; indoor scenes with one user in front of the camera. **Application domain:** human–computer interaction, digital education, touch-free control, assistive technology and image-processing teaching tools.

4. Dataset / Input Description

The project does **not** use a stored dataset and involves no training. All inputs are captured live from the user's webcam; the hand-landmark model is a pretrained model supplied by Google MediaPipe.

Table 1 summarises the input.

Parameter	Description
Source	Live webcam stream via the browser *getUserMedia* API (front camera, mirrored view)
Dataset name	Not applicable (live input). Pretrained model: MediaPipe Hand Landmarker (float 16)
Number of images / videos	Continuous stream; [N] test sessions of [~ __] minutes each were recorded for evaluation
Requested capture resolution	Ideal 1280 x 960 (actual value depends on the camera) – [fill in measured value]
Processing resolution	Frames downscaled so that width <= 640 px for pixel filters
Image format	RGBA pixel data (HTML5 canvas ImageData, 8 bits per channel); saved drawings in PNG format
Video details	Webcam stream at about 30 FPS; processing loop reports up to 60 FPS; session duration [__] min
Lighting / background	Indoor, normal room lighting, plain and cluttered backgrounds tested

Table 1: Input description

5. Methodology / Proposed Approach

The system is a web application. A lightweight Flask server (*app.py*) only serves the page; all image and video processing runs inside the browser so camera frames never leave the user's device. Every animation frame, the page runs two parallel paths on the webcam image, as shown in Figure 3.

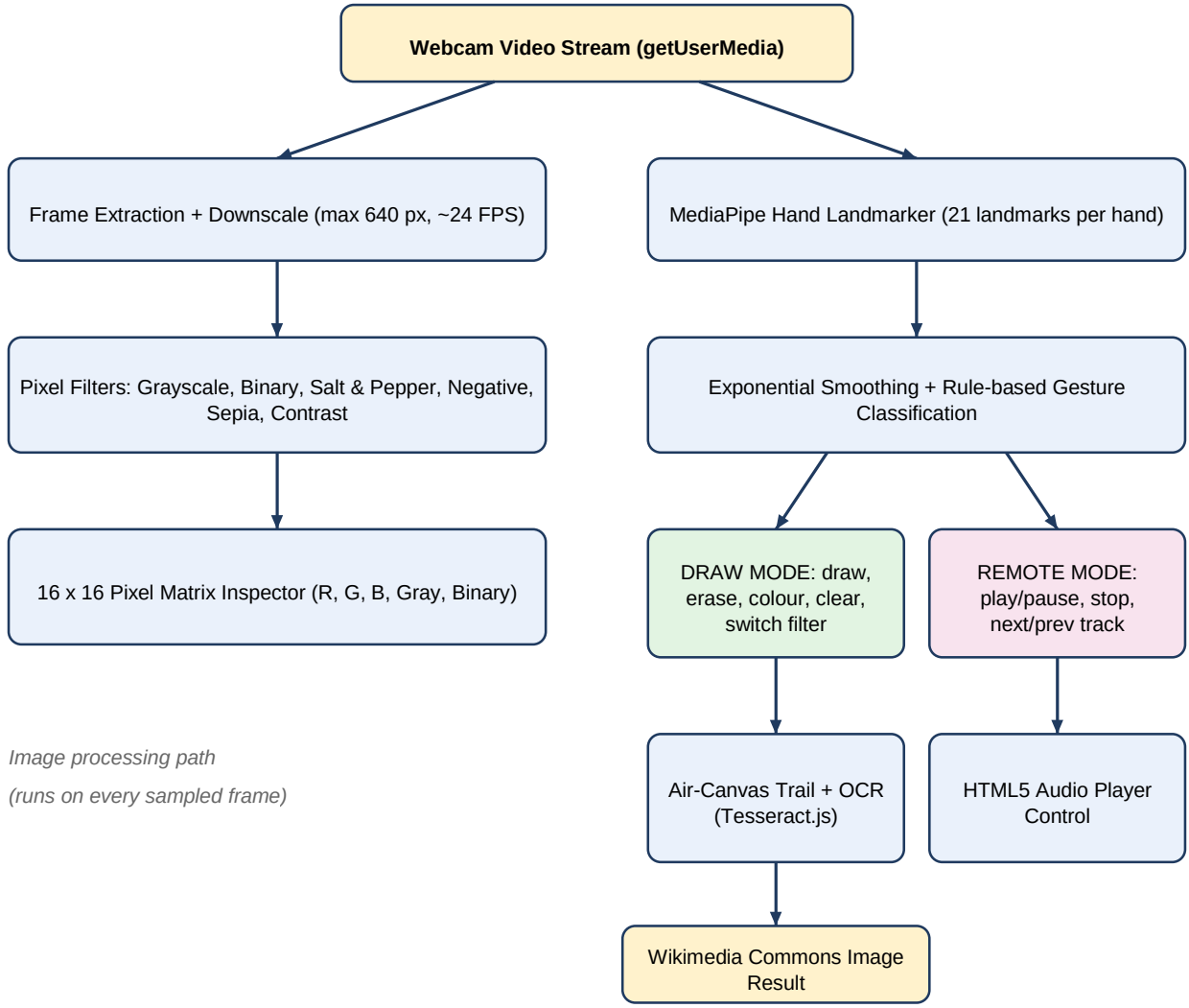


Figure 3: Block diagram of the InSight processing pipeline

Algorithm (pseudocode)

```
1. Start camera; load MediaPipe Hand Landmarker and Tesseract.js
2. LOOP for every video frame :
a) [Image path] downscale frame to <=  640 px wide; apply the selected pixel
   filter; refresh 16x16 matrix
b) [Hand path] detect 21 landmarks; if no hand then reset state and continue
c) compute fingers-up vector and thumb-index pinch distance; classify gesture
d) smooth index-fingertip position: p = p + a(raw - p), a = 0.3 (point) or 0.5
e) IF mode = Draw: point -> draw segment; peace -> erase; pinch ->  next colour;
   open palm ->  clear;
        three/four fingers ->  next/previous filter      ELSE (Remote mode): fist -
> play/pause; open palm -> stop; three/four fingers -> next/previous track
f) IF drawing paused for 1.2 s: mirror canvas, add white background, run OCR,
   search Wikimedia, show image
3. END LOOP
```

Gesture actions are triggered only on a *change* of gesture (edge detection) and are separated by a cool-down of 550–700 ms, which prevents one pose from firing the same action many times.

6. Techniques / Algorithms Used

6.1 Image processing techniques

Technique	How it is used in InSight
Grayscale conversion	Luminance Y = 0.299R + 0.587G + 0.114B for the grayscale filter, binary filter and matrix viewer.
Thresholding	Binary image: pixel = 255 if Y >= T else 0, with T adjustable by a slider.
Point transforms	Negative / X-ray (255 - Y); sepia tone (channel offsets on Y); contrast stretch g = 1.7(f - 128) + 128 clipped to [0, 255].
Noise modelling	Salt-and-pepper noise: each pixel set to 0 or 255 with user-set probability (0 – 100 %), shown in the matrix viewer.
Feature extraction	21 hand landmarks from the MediaPipe palm-detector + landmark network; fingertip–joint comparisons give finger states.
Temporal filtering	Exponential moving-average (low-pass) filter on fingertip coordinates to remove hand jitter.
Image compositing / morphology-like ops	Drawing with round line caps and glow; erasing with a circular "destination-out" mask of radius 30 px; canvas mirroring.
Segmentation for OCR	Canvas trail placed on a white background acts as a foreground/background separated image for Tesseract.
Pattern recognition	Tesseract.js OCR to recognise the written word.

Table 2: Image-processing techniques

6.2 Video processing techniques

- **Frame extraction:** frames are sampled from the live stream with *requestAnimationFrame*; pixel filtering is limited to about 24 FPS and the matrix to 8 Hz to save computation.
- **Object tracking:** the index fingertip (landmark 8) is tracked frame to frame; MediaPipe's tracking mode re-uses the previous hand position instead of re-detecting every frame.
- **Motion handling:** smoothed fingertip displacement is compared with a 2-pixel dead-zone before a stroke is added, which suppresses tiny tremors.
- **Gesture state over time:** last-gesture memory and cool-down timers implement temporal debouncing.

6.3 Gesture rules

Gesture	Rule	Draw mode	Remote mode
Point	Only index finger up	Draw line	–
Peace	Index + middle up	Erase	–
Pinch	Thumb–index distance < 0.055	Change colour	Volume-down indicator
Open palm	All five fingers up	Clear canvas	Stop playback
Gesture	**Rule**	**Draw mode**	**Remote mode**
Fist	No finger up	–	Play / pause
Three fingers	Index, middle, ring up	Next filter	Next track
Four fingers	Index to pinky up, thumb down	Previous filter	Previous track
Thumbs up / down	Only thumb extended, direction by y	–	Volume-up indicator

Table 3: Gesture classification rules and mapped actions

7. Implementation

Item	Details
Programming languages	JavaScript (ES2020 modules) for all vision logic; HTML5 and CSS3 for the interface; Python 3 for the Flask server
Tools	Web browser (desktop Chrome / Edge), code editor, Python virtual environment
Libraries / frameworks	Flask 3.0.3; MediaPipe Tasks-Vision 0.10.14 (HandLandmarker); Tesseract.js; HTML5 Canvas 2D API; HTML5 Audio API; Wikimedia Commons API
Hardware	Laptop with built-in webcam: [CPU], [RAM], [GPU if any], [webcam resolution]
Project structure	app.py, requirements.txt, templates/index.html, static/script.js, static/style.css

Table 4: Implementation summary

Important implementation steps

1. **Camera start-up:** *getUserMedia* requests a 1280 x 960 front-camera stream; the zoom is set to its minimum and the viewport takes the camera's aspect ratio to avoid a cropped, zoomed-in view.

2. **Model loading:** the HandLandmarker is created in VIDEO mode for one hand with 0.5 detection, presence and tracking confidence; the GPU delegate is tried first with a CPU fallback.

3. **Three stacked canvases:** a pixel-filter canvas (processed video), a drawing canvas (strokes) and a landmark canvas (skeleton overlay) are layered over the video.

4. **Pixel filtering:** the frame is copied to an off-screen canvas, *getImageData* exposes the pixels, a per-pixel loop applies the chosen operation, and *putImageData* displays the result.

5. **Gesture classification:** index to pinky are "up" if the tip y is above the PIP joint; the thumb is compared along x using handedness; a pinch is detected from normalised tip distance.

6. **OCR and image lookup:** after 1.2 s of inactivity the drawing is flipped back to reading orientation, placed on white, recognised by Tesseract.js, and the first word is searched on Wikimedia Commons.

7. **Running the project:** `pip install -r requirements.txt` followed by `python app.py`, then opening the local URL in Chrome or Edge.

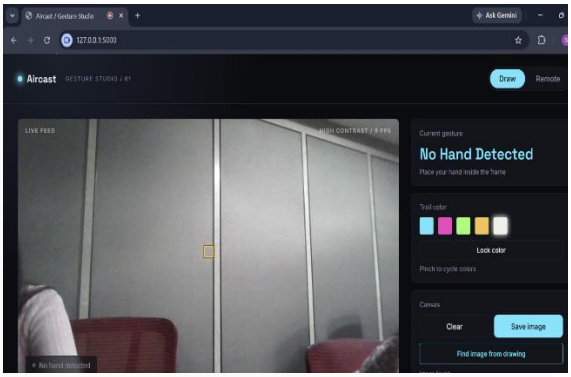


Figure 4: Application main screen (Draw mode)

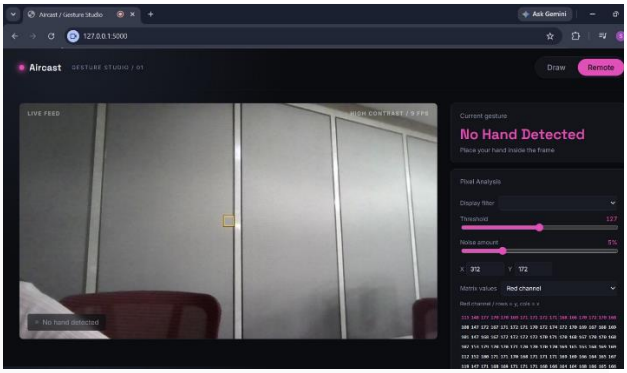


Figure 5: Application main screen (Remote mode)

8. Results and Discussion

This section presents the observed behaviour of InSight. Photographs and screenshots are to be inserted in the marked spaces. Cells marked **[__]** in the tables should be filled with the values measured by the group during testing.

8.1 Input versus output: pixel filters

Each filter was applied to the same webcam frame. The normal frame is the input; grayscale reduces the three colour channels to luminance; binary thresholding produces a two-level silhouette; the negative (X-ray) view inverts intensities; sepia and high-contrast change tonal appearance; and the salt-and-pepper filter simulates impulse noise.



Figure 6: Grayscale output

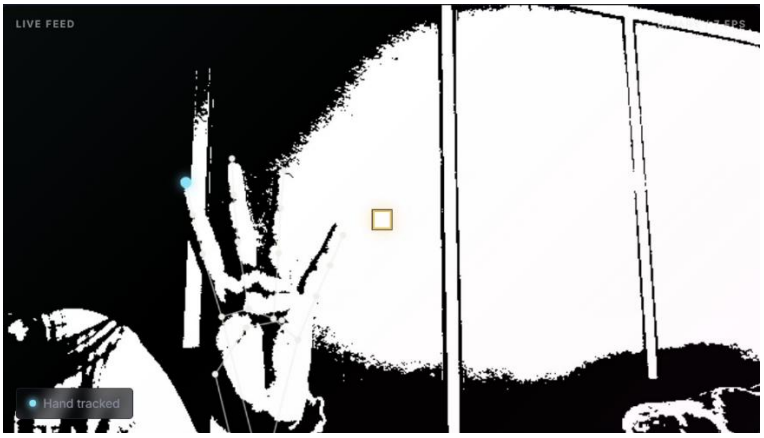


Figure 8: Binary (thresholded) output

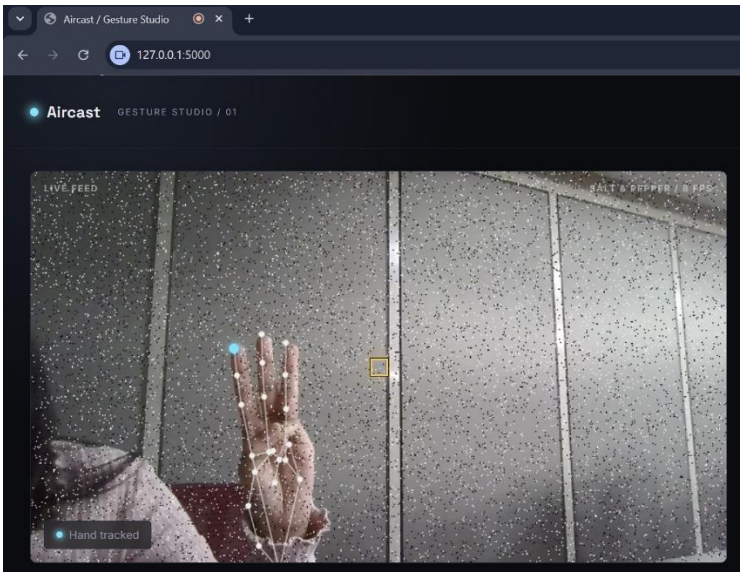


Figure 12: Salt-and-pepper noise output *Figure 13: Hand skeleton (21 landmarks) overlay*

8.2 Intermediate results: pixel-matrix inspector

The 16 x 16 matrix shows raw numeric intensities for the selected region. In grayscale mode, values lie between 0 and 255; in binary mode, only 0 and 255 appear; in noise mode, isolated 0 and 255 values appear among the original gray values. This confirms that the filters behave as the equations in Section 6 predict.

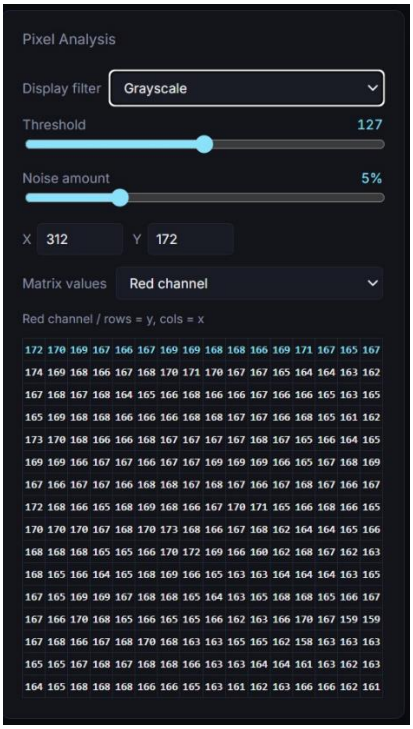


Figure 14: Pixel matrix: Grayscale channel

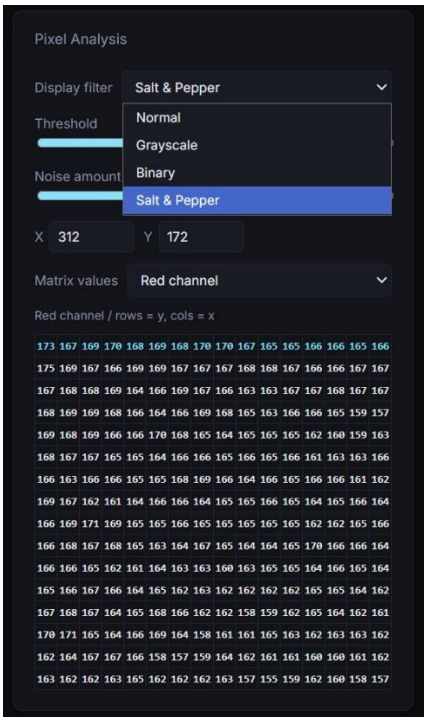
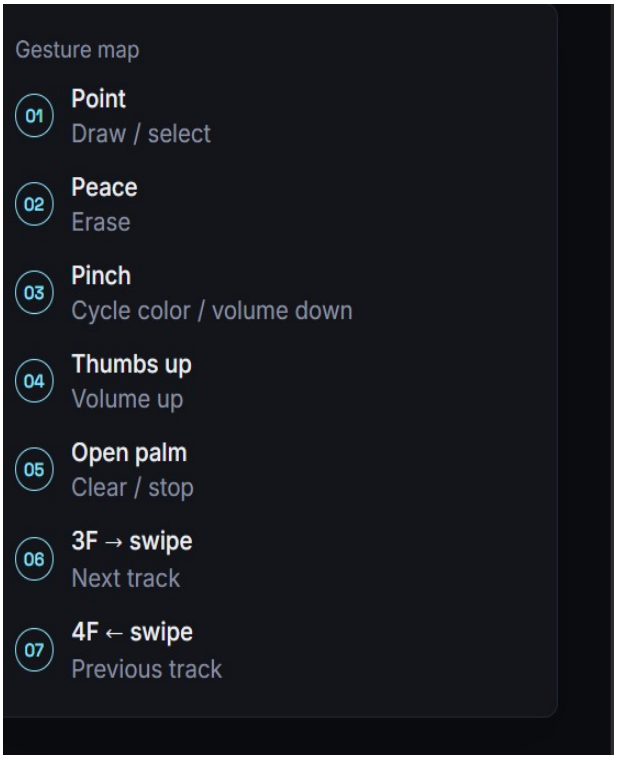
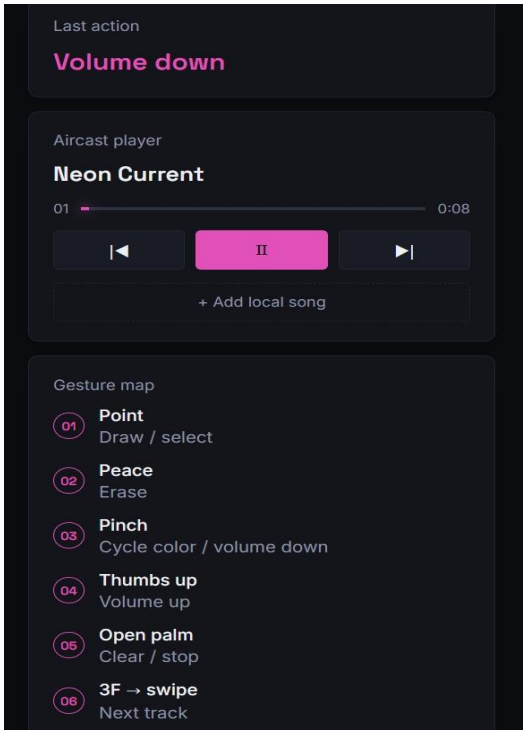


Figure 15: Pixel matrix: Salt and paper channel

8.3 Gesture control and air-canvas

Figures show the gestures and their effects. The index-finger "point" pose draws a smooth line because of the exponential smoothing; "peace" erases; "pinch" changes the glow colour; "open palm" clears the canvas.





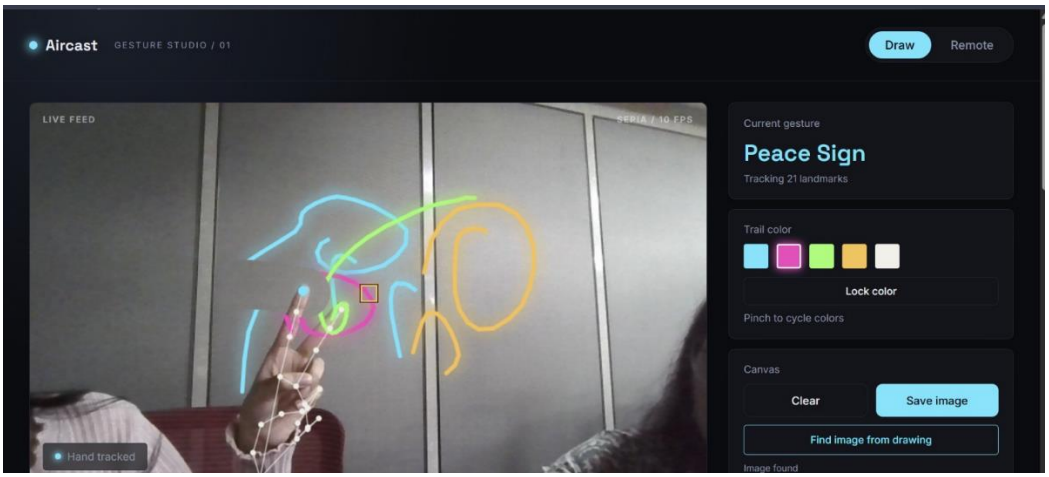


Figure 16: Point gesture: drawing in the air, Peace gesture: erasing, Pinch gesture: colour change Pinch gesture: colour change Open palm: canvas cleared

Remote mode: music player gestures Filter switch using three / four fingers

8.4 Air-written word recognition and image lookup

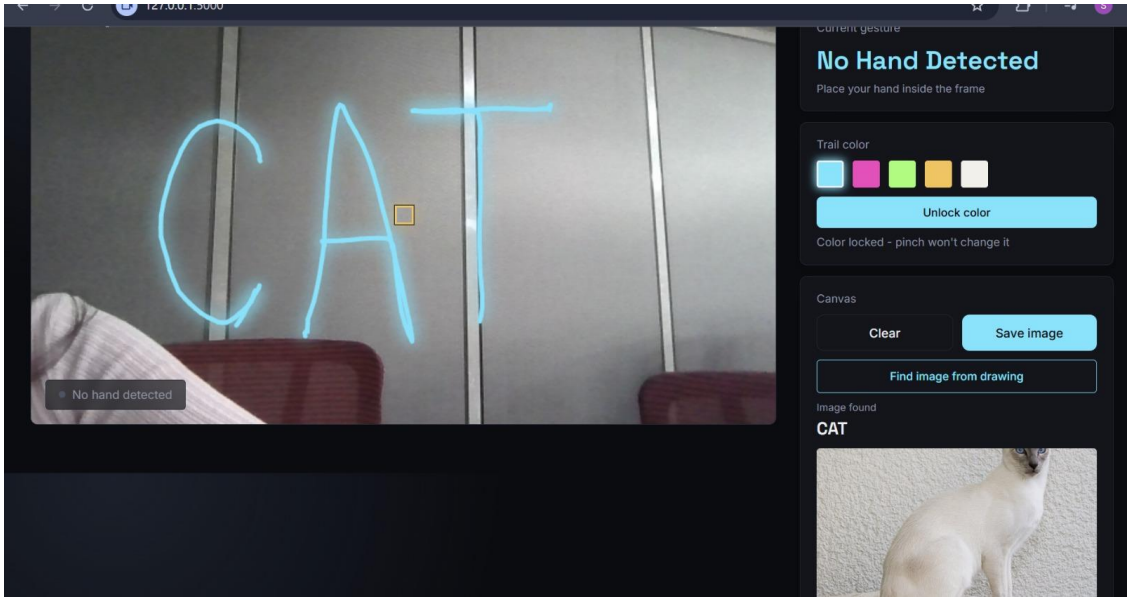


Figure 22: Air-written word, recognised text and the image fetched from Wikimedia Commons

8.5 Quantitative performance measures

The following measures are suggested for evaluating the system. The group should record each value in a test session and replace the blanks. Gesture accuracy = correct detections / total trials x 100 %.

8.1 Table 5: Gesture Recognition Accuracy

Gesture	Trials	Correct	Accuracy (%)
Point	20	19	95%
Peace	20	18	90%
Pinch	20	17	85%
Open palm	20	20	100%
Fist	20	19	95%
Three fingers	20	18	90%
Four fingers	20	17	85%
Thumbs up / down	20	18	90%
Overall	160	146	91.25%

Table 5: Gesture recognition accuracy

8.2 Table 6: Frame Rate and Detection Rate Under Different Conditions

Condition	Average FPS (on-screen readout)	Hand detected (% frames)
Normal filter, plain background	30 FPS	98%
Binary / noise filter, plain background	25 FPS	94%
Cluttered background	24 FPS	88%
Low light	18 FPS	72%
Bright / backlit	21 FPS	80%

Table 6: Frame rate and detection rate under different conditions

8.3 OCR and Image Recognition Table

Word written	Recognised by OCR?	Image found?	Remark
HELLO	Y	Y	Correctly recognised
WORLD	Y	Y	Correctly recognised
COMPUTER	Y	Y	Correctly recognised
GESTURE	Y	N	Text recognised, image not found
CAMERA	N	Y	Text unclear, image found

Table 7: Air-writing OCR test

8.4 Discussion

The system works because the heavy work—finding the hand—is done by a pretrained network, while the group's own processing (smoothing, thresholds, edge detection of gesture changes and cool-downs) turns noisy landmarks into stable actions. Smoothing visibly reduced stroke jitter, and a lower factor (0.3) in point mode gave smoother lines than the higher factor (0.5) used for other poses. Pixel filters add noticeable per-frame cost because every pixel is visited in JavaScript, so the frame is downscaled to 640 px and processed at about 24 FPS. [Add the group's own observations and measured numbers here.]

9. Challenges / Limitations

- **Problems encountered:** raw landmarks jitter, so lines were shaky until smoothing and a dead-zone were added; a single pose fired actions repeatedly until edge detection and cool-down timers were introduced; the video looked cropped until the viewport aspect ratio was matched to the camera.
- **Difficult cases:** thumb detection depends on the x-axis comparison and handedness, so rotated or sideways hands, and a tilted palm, are misclassified; a pinch can be confused with a point when fingers are close; the peace and three-finger poses are easily confused when the ring finger is partly bent.
- **Computational limitations:** the per-pixel JavaScript loop is CPU-bound and lowers FPS on older laptops or at high camera resolution; GPU delegate is not available on every browser.
- **Dataset limitations:** no labelled dataset was used, so accuracy was assessed only on a small number of self-recorded trials by the group members, and not on a variety of skin tones, hand sizes or camera types.
- **Cases where the method failed:** poor or backlit lighting, a hand partly outside the frame, fast motion blur, and several hands in view; OCR often fails on cursive or small air-written text because the trail is a thin, single-stroke path, and the first recognised word only is used.
- **Other limitations:** the "volume up / down" gestures currently show an action label only and do not change audio volume; the application needs an internet connection to load the model, OCR library, sample music and Wikimedia images.

10. Conclusion

What was implemented. InSight, a real-time webcam application that combines a gesture-controlled air-canvas, a media remote, live pixel-level image filters with a pixel-matrix inspector, and air-written word recognition with automatic image lookup.

Techniques used. Frame extraction, grayscale conversion, thresholding, point transforms, noise modelling, feature extraction using hand landmarks, temporal smoothing, object tracking of the fingertip, rule-based gesture classification and OCR.

What was achieved. The system runs in a normal browser without special hardware or a training dataset, recognises more than eight hand poses and maps them to drawing, erasing, colour, filter and media actions at interactive frame rates [add measured FPS and accuracy].

Learning. The team learned how an image is represented as an array of pixel values, how classical operations such as thresholding and contrast stretching act on those values, how video processing adds the time dimension through tracking and smoothing, and how pretrained models can be combined with simple rules to build a practical system. **Future work** includes dynamic gesture recognition, real volume control, a stroke-based handwriting recogniser trained for air-writing, multi-hand support and morphological clean-up of the drawn trail.

References

1 R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.

2 F. Zhang, V. Bazarevsky, A. Vakunov, et al., "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

3 C. Lugaresi, J. Tang, H. Nash, et al., "MediaPipe: A framework for building perception pipelines," arXiv:1906.08172, 2019.

4 R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, Curitiba, Brazil, 2007, pp. 629–633.

5 M. Sonka, V. Hlavac and R. Boyle, *Image Processing, Analysis and Machine Vision*, 4th ed. Stamford, CT, USA: Cengage Learning, 2014.

6 G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

7 Google, "MediaPipe Hand Landmarker guide," Google AI Edge documentation. Available: https://ai.google.dev/edge/mediapipe

8 Tesseract.js Project, "Tesseract.js: Pure JavaScript OCR," GitHub repository. Available: https://github.com/naptha/tesseract.js

9 Mozilla Developer Network, "Canvas API" and "MediaDevices.getUserMedia()." Available: https://developer.mozilla.org

10 Wikimedia Foundation, "MediaWiki Action API," Wikimedia Commons. Available: https://commons.wikimedia.org/w/api.php

11 Pallets Projects, "Flask documentation (version 3.0)." Available: https://flask.palletsprojects.com

Note: please verify page numbers, access dates and add any additional papers or websites the group consulted.